

DAY 5 PRACTICAL TASK

Complete RAG Question Answering API with LLM Integration

Course: Applied RAG Engineering	Mode: Theory + Practical
Focus: Retrieval + Prompt + LLM	Backend: FastAPI

Project Objective

The purpose of Day 5 is to complete the RAG pipeline by connecting semantic retrieval with an LLM. Students will accept a user question, retrieve the most relevant chunks from the vector database, build a grounded prompt, send the prompt to an LLM, and return the generated answer together with source information.

This practical represents the **answer generation stage** of a Retrieval-Augmented Generation system.

1. Day 4 Recap

- Document embeddings were generated.
- Embeddings and metadata were stored in a vector database.
- A user question was converted into a query embedding.
- Similarity search returned top-k relevant chunks.

2. Complete RAG Learning Flow

```
User Question
↓
Generate Query Embedding
↓
Vector Search
↓
Retrieve Top-K Chunks
↓
Build Context
↓
Create Prompt
↓
Send Prompt to LLM
↓
Generate Final Answer
↓
Return Answer + Sources
```

3. What Happens After Retrieval?

- The retrieved chunks become the context for the LLM.
- Only the most relevant chunks should be included in the prompt.
- The model should be instructed to answer using the provided context.
- Source information should be preserved so the final response can show where the answer came from.

4. Complete RAG Architecture

```
DOCUMENT SIDE

Documents
↓
Preprocessing
↓
Chunking
↓
Embeddings
↓
Vector Database

QUESTION SIDE

User Question
↓
Query Embedding
↓
Retriever
```

↓
Relevant Context
↓
Prompt
↓
LLM
↓
Answer + Sources

5. Context Building

- Collect the top retrieved chunks.
- Combine them into a clean context block.
- Keep source metadata for each chunk.
- Avoid including too much irrelevant text.
- Prepare the context for prompt construction.

6. Prompt Construction

A simple grounded prompt may follow this pattern:

```
Context:
{retrieved_chunks}

Question:
{user_question}

Instruction:
Answer the question using only the provided context.
If the context does not contain enough information,
say that the available documents do not contain enough information.
```

The goal is to reduce unsupported answers and keep the response grounded in retrieved content.

7. LLM Integration

- The LLM receives the final prompt.
- It uses the retrieved context to generate a natural-language answer.
- The project may use an API-based or local LLM, depending on the classroom setup.
- API keys should be stored in environment variables rather than written directly in source code.

8. Create rag_service.py

The RAG service should coordinate the main question-answering flow.

- Receive the user question.
- Retrieve relevant chunks.
- Build the context.
- Create the prompt.
- Call the LLM.
- Return the final answer and sources.

9. Main Endpoint

Create the following endpoint:

```
POST /ask
```

Example Request

```
{
  "question": "What is FastAPI?",
  "top_k": 3
}
```

Expected Processing

```
Question
↓
Generate Query Embedding
↓
Search Vector Database
↓
Retrieve Top-K Chunks
↓
Build Context
↓
Create Prompt
↓
Call LLM
↓
Return Answer + Sources
```

Example Response

```
{
  "question": "What is FastAPI?",
  "answer": "FastAPI is a Python framework used for building APIs.",
  "sources": [
    "sample.pdf"
  ]
}
```

```
]
}
```

10. Source-Aware Response

The final API response should preserve enough information to identify the document sources used for the answer.

- Source file name
- Chunk ID when available
- Source list
- Optional retrieved text preview

11. No Relevant Context Case

If the retrieved context does not contain useful information, the system should return a controlled answer.

```
{
  "answer": "The available documents do not contain enough information to answer this question.",
  "sources": []
}
```

12. Error Handling

- Empty question
- Invalid top_k value
- Vector database not initialized
- No matching chunks
- Embedding generation failure
- LLM/API failure
- Missing environment variable or API key

13. Recommended Project Structure

```
day5_complete_rag_api/  
■  
■■■ app/  
■ ■■■ __init__.py  
■ ■■■ main.py  
■ ■■■ routes.py  
■ ■■■ schemas.py  
■ ■■■ file_loader.py  
■ ■■■ text_cleaner.py  
■ ■■■ text_splitter.py  
■ ■■■ embeddings.py  
■ ■■■ vector_store.py  
■ ■■■ search_service.py  
■ ■■■ prompt_builder.py  
■ ■■■ rag_service.py  
■  
■■■ data/  
■  
■■■ vector_db/  
■  
■■■ .env  
■■■ requirements.txt  
■■■ README.md
```

14. File Responsibilities

main.py

Creates and configures the FastAPI application.

routes.py

Contains the /ask endpoint and receives API requests.

schemas.py

Defines request and response models for the RAG endpoint.

embeddings.py

Generates query embeddings and document embeddings.

vector_store.py

Loads and searches stored vectors.

search_service.py

Retrieves the top-k relevant document chunks.

prompt_builder.py

Combines context, instructions, and the user question into a final prompt.

rag_service.py

Coordinates retrieval, context building, prompt construction, LLM calling, and final response creation.

.env

Stores environment variables such as API keys when an external LLM provider is used.

15. Complete Internal Architecture

```
POST /ask
↓
routes.py
↓
schemas.py
↓
rag_service.py
↓
search_service.py
↓
embeddings.py
↓
vector_store.py
↓
Top-K Relevant Chunks
↓
prompt_builder.py
↓
LLM
↓
Answer + Sources
↓
JSON Response
```

16. Testing Requirements

Test 1 - Normal Question

```
{
  "question": "What is FastAPI?",
  "top_k": 3
}
```

Test 2 - RAG Question

```
{
  "question": "What is Retrieval-Augmented Generation?",
  "top_k": 3
}
```

Test 3 - Smaller top_k

```
{
  "question": "How are embeddings used?",
  "top_k": 1
}
```

Test 4 - Irrelevant Question

```
{
  "question": "What is the weather today?",
  "top_k": 3
}
```

Expected behavior: return a controlled response if the indexed documents do not contain the answer.

Test 5 - Empty Question

```
{
  "question": "",
  "top_k": 3
}
```

Expected behavior: return a validation or meaningful error response.

17. Suggested Tools and Libraries

- Python
- FastAPI
- Uvicorn
- Pydantic
- sentence-transformers
- FAISS or Chroma
- python-dotenv
- An API-based or local LLM

18. Learning Outcomes

- Understand the complete RAG question-answering flow.
- Use retrieved chunks as LLM context.
- Build a grounded prompt.

- Integrate an LLM with FastAPI.
- Return answer and source information.
- Handle no-context and failure cases.
- Understand the role of top-k retrieval in answer generation.
- Separate retrieval, prompt, and RAG orchestration logic.
- Test the complete RAG API using Swagger UI or Postman.

19. Student Practical Task

Project Title: Complete RAG Question Answering API

Build a FastAPI-based RAG system that accepts a user question, retrieves the most relevant document chunks from the vector database, builds a context-aware prompt, sends the prompt to an LLM, and returns the final answer together with its sources.

20. Final Day 5 Result

```
Question
↓
Embedding
↓
Vector Search
↓
Top-K Chunks
↓
Context
↓
Prompt
↓
LLM
↓
Answer + Sources
```

At the end of Day 5, students should have a working **end-to-end RAG question-answering API**.